

Building a MCP using C# (.NET)

AI Usage Notes

1. Project Overview

- Organizations today keep huge amounts of company information in database form, which complicates their retrieval for non-technical personnel.
- Our goal was to develop an AI-based solution that would enable employees to ask queries in natural language and get answers from the company's databases.

We decided to use:

- **MCP (Model Context Protocol)** as a protocol that would serve as a standard interface between AI solutions and enterprise systems.
- **Claude Desktop** as an AI client capable of finding and using MCP components.
- **SQLite** is a database management system convenient for fast development cycles.
- **C# .NET** (version 8) for implementing the MCP Server application.

2. AI-Assisted Development Journey:

- **First Approach** - We began the process by giving the following simple instruction in Codex: "Develop an MCP Server using C#, SQLite, and Claude Desktop." While the resulting code met technical criteria, it wasn't entirely aligned with the business requirements we had envisioned.

3. Challenges Encountered:

- Poor comprehension of the MCP architecture initially.
- Incomplete structure of the tool with no business logic.
- Generic database design was insufficient.
- There were no security and access controls implemented.
- Manual integration with the Claude Desktop was needed.
- Iteration was needed for refinement of the produced code.

4. The Failure of AI:

In the course of our design, there were several limitations encountered, namely:

- Produced generic code in place of industry solutions.
- Role-Based Access Control was not considered at first.
- Employee-sensitive data were left unprotected.
- Integration of Claude Desktop would still have to be manually debugged.
- Validation for configuration path and deployment was needed.
- Automatic detection of tools and their registration did not occur.

5. Key Insights

- AI enhances development, yet does not substitute for engineering judgments.
- Clear instructions yield better outcomes.
- Human oversight is always needed.
- MCP development demands knowledge of AI, its software, databases, and integration.
- The security aspect should be considered from the very beginning.
- Iterative prompting yields better outcomes.

6. Outcomes:Successfully achieved:

Development of MCP Server in C#.NET (version 8)	✓
Development of SQLite database	✓
Development of MCP Tools to extract business information	✓
Integration with Claude Desktop	✓
Knowledge of Security/Sensitive Data Protection principles	✓
Knowledge of RBAC Control	✓
AI-assisted development process	✓